



ENGINEERING PROPOSAL

Training a Car to Race, Overtake, and Win

A staged refactor of the current single-car genetic training system into a multi-car race environment with collision avoidance, tactical passing, and race-outcome fitness.

RACING SIMULATION WITH NN + GA

Architecture, observations, collision model, rewards, curriculum, self-play, and implementation roadmap



EXECUTIVE SUMMARY

Recommended direction

Evolve only the candidate car. Race it against fixed scripted opponents first, then against frozen champion policies. This keeps fitness comparable while the candidate learns collision avoidance, passing, defending, and winning.

The current simulator is designed around one vehicle. Its neural observation describes the ego car and track geometry, lidar sees only track boundaries, and each genome runs alone. The required change is therefore broader than adding a collision penalty: the environment, observation, evaluator, and fitness definition must all become race-aware.

The recommended implementation sequence is:

- 1 Create a multi-car world**
Shared track, per-car state, simultaneous action evaluation
- 2 Make opponents observable**
Relative position, speed, heading, presence, and car-aware lidar
- 3 Add collision detection**
Oriented vehicle bodies checked inside physics substeps
- 4 Train against controlled traffic**
Scripted pace cars before archived neural champions
- 5 Optimize for race results**
Win rate and finishing position dominate dense shaping rewards

1 / ARCHITECTURE

Move from RacingEnv to a shared race world

Today, RacingEnv owns one CarState plus one set of controller filters and history. For a race, the track and timing system should be shared while dynamic state belongs to each participant.

Current design	Race-aware design
One CarState in RacingEnv	RaceEnvironment owns an array of RaceCar
History and controller state are global	Each RaceCar owns its own history and controller runtime
One action followed by one physics step	Observe all cars, compute all actions, then advance the world
Lap completion terminates the episode	Race continues until the candidate finishes, crashes, or times out

Suggested data split

RaceEnvironment Shared track and parameters Race clock and scenario seed cars[MAX_RACE_CARS]	RaceCar CarState Controller filters Observation history Active / crashed / finished Race position
---	--

Required frame order

- Generate observations from one immutable world snapshot.
- Compute every car's action before any car moves.
- Integrate vehicle motion for the frame.
- Resolve track status, vehicle collisions, timing, and race position.
- Calculate the candidate car's reward and termination state.

Simultaneous action calculation matters: moving cars sequentially would give later cars fresher information and create an artificial ordering advantage.

2 / PERCEPTION

Teach the policy to see traffic

The existing observation contains ego dynamics, history, boundary lidar, and track lookahead. It cannot distinguish an empty circuit from one occupied by another car. Add explicit relative features for the four closest relevant opponents.

Feature per opponent	Meaning	Suggested normalization
Presence	Whether this observation slot is occupied	0 or 1
Longitudinal gap	Distance ahead or behind along the circuit	Clamp by sensor range
Lateral gap	Opponent offset in the ego car frame	Divide by track half-width
Relative longitudinal speed	Closing or opening rate	Divide by maximum speed
Relative lateral speed	Cross-track motion	Divide by a lateral-speed limit
Relative heading	Opponent orientation difference	Divide by pi

Four slots times six values adds 24 NN inputs. NN_INPUT_COUNT, NN_GENOME_COUNT, serialization validation, and the saved-network magic version must change together. Old saved networks should be rejected rather than interpreted with the new topology.

Car-aware lidar

Extend the existing rays so they intersect vehicle bodies as well as track boundaries. Lidar supplies close-range shape and clearance; explicit opponent features supply motion and identity-independent tactical context. Both are useful.

Collision model

Give each vehicle a half-length and half-width and detect overlap between oriented rectangles with a separating-axis test. Run the check inside the existing physics substeps to reduce tunneling at high relative speed.

Initial behavior	Later refinement
Candidate collision: large penalty and episode termination	Impulse response, spins, and damaged-car motion
Opponent collision: mark involved cars inactive	Race control, recovery, or retirement behavior
Track departure remains a separate failure	Distinguish wall, grass, and recoverable excursions

3 / OPPONENTS

Build traffic before building self-play

Keep the candidate genome as the only evolving policy during evaluation. Opponents remain fixed for a generation so candidate fitness is stable and thread-safe.

Phase A: scripted drivers

- A slow centerline pace car that establishes the basic passing task.
- A car holding a constant lateral line, forcing inside or outside decisions.
- A car with controlled lane changes and randomized target speed.
- Multiple opponents with reproducible speed, gap, and lane variation.

Phase B: frozen neural champions

Archive strong policies in a hall of fame. Sample opponents from that archive for each scenario, but do not update those policies while the generation is running. This provides increasingly capable opposition without making the objective move during selection.

Do not evaluate one evolving population as a single 64-car race. Results would depend heavily on grid position and opponent composition, and each genome's fitness would change when unrelated genomes change.

Keep the current action abstraction

The existing outputs - desired speed, lateral offset, and heading offset - are a good tactical interface. The policy can select an inside or outside line while the vehicle controller converts that intent into throttle, brake, and steering. Direct actuator control is not required for the first multi-car milestone.

4 / OBJECTIVE

Make winning dominate the fitness function

Dense progress rewards are necessary while policies are weak, but the final ordering of genomes must be controlled by race outcomes. Otherwise the GA can produce a fast hot-lap car that is poor in traffic.

Event	Initial value	Purpose
One lap of forward progress	+10	Dense learning signal
Clean pass	+2	Encourage decisive overtaking
Being passed	-2	Encourage defending and pace
Finish the race	+10	Reward reliability
Finishing-position score	0 to +20	Differentiate completed races
Win	+30	Make the final objective explicit
Car collision	-20 and terminate	Strongly reject unsafe driving
Leave the track	-10 and terminate	Preserve track discipline

Recommended priority

Win rate > finishing position > race completion > progress > smoothness. A practical late-stage formula is:

```
fitness = 50 * win_fraction + 20 * finish_fraction
+ 10 * normalized_position_score + progress_reward - penalties
```

Prevent reward exploits

- Reward net position gain or one pass per opponent per lap; do not reward repeated position swapping.
- Use a small near-contact penalty, if any. A large proximity penalty can teach the car never to attempt a pass.
- End the candidate episode after a crash, but retain the exact collision and race-state telemetry.
- Use race result as the strongest signal once policies reliably finish.

5 / CURRICULUM

Increase traffic complexity only after competence

A full grid gives early random policies mostly collisions and weak credit assignment. Introduce one new tactical demand at a time and advance using measured performance.

Stage	Skill	Environment	Promotion signal
1	Obstacle awareness	One stationary or very slow car	Avoid collision in 90% of runs
2	Straight-line pass	One slower moving car on a straight	Pass cleanly in 80% of runs
3	Corner traffic	One slower car through varied corners	Finish without contact in 80%
4	Traffic choice	Two cars with randomized lines and speeds	Positive average position gain
5	Short race	Four-car rolling-start races	Finish rate above 80%
6	Competitive race	Full grid against archived champions	Win rate becomes primary

Replace solo sector episodes with race scenarios

- Candidate behind a slower car.
- Candidate side-by-side with another car.
- Candidate defending against a faster car.
- Candidate starting first and managing pressure.
- Candidate starting last and navigating traffic.
- Randomized grid race for generalization.

Every genome in a generation should receive the same scenario seeds, opponent policies, and starting positions. This makes fitness differences reflect policy quality rather than luck.

6 / DELIVERY PLAN

Implement in testable milestones

Milestone 1 - Two-car proof of concept

- Introduce RaceEnvironment and per-car runtime state.
- Render one candidate and one scripted slower opponent.
- Add opponent observation slots and car-aware lidar.
- Terminate on collision and reward one clean pass.

Milestone 2 - Scenario training

- Add deterministic scenario seeds and randomized traffic parameters.
- Track position, passes, collisions, finishes, and wins.
- Replace sector-only fitness with mixed passing and short-race scenarios.

Milestone 3 - Competitive opponents

- Archive champion policies and sample a frozen opponent set.
- Introduce multi-car rolling starts and finishing-position rewards.
- Tune fitness so win rate dominates after reliable completion.

Computational budget

A configuration with 64 genomes, six scenarios, five cars, and 12,000 steps can approach 23 million car steps per generation. Keep the cost controlled with scripted opponents, early termination after crashes, small grids during the curriculum, and extra evaluation only for promising genomes. Preserve the thread pool's two-processor reservation.

Code area	Primary change
code/racing.h	Multi-car structures, opponent inputs, collision and race metrics
code/simulation.cpp	Per-car stepping, simultaneous world step, car sensing, collisions, positions
code/training.cpp	Race scenarios, opponent controllers, outcome fitness, common seeds
code/rendering.cpp	Render all cars, collisions, positions, passes, and race status
code/main.cpp	Initialize race participants and expose training/race modes

CONCLUSION

The first useful target

Build a two-car simulation with one scripted slower opponent, opponent-aware observations, collision termination, and a clean-pass reward.

That milestone validates the complete tactical learning loop: sensing another vehicle, choosing a passing line, controlling closing speed, avoiding contact, and completing the pass. Once it is reliable, additional cars, randomized scenarios, archived champions, and win-dominant fitness can be layered on without changing the core architecture.

Definition of success for the first milestone

- The candidate detects and identifies a slower car ahead.
- It selects a feasible lateral passing line before reaching collision distance.
- It modulates speed when a safe pass is not available.
- It completes a pass without contact or leaving the track.
- The same policy succeeds across randomized gaps, speeds, and lateral positions.

Prepared for the racing project - September 2026